

Peter Dimov (pdimov@pdimov.com)
Louis Dionne (ldionne.2@gmail.com)
Nina Ranns (dinka.ranns@gmail.com)
Richard Smith (richard@metafoo.co.uk)
Daveed Vandevoorde (daveed@edg.com)

More constexpr containers

R0: Original proposal, presented in Albuquerque 2017.

EWG approved the direction: SF: 11 | F: 12 | N: 2 | A: 0 | SA: 0

R1: Provided initial wording.

EWG approved the proposal as presented: SF: 23 | F: 11 | N: 0 | A: 0 | SA: 0

R2: Implemented core review comments.

R3: Extends proposal for non-transient allocations.

R4: Tighten wording for "construct" and "destruct".

R5 (this revision):

mark_immutable_if_constexpr to address some recently found issue.

Remove prohibition on virtual destructors (i.e., merge in P1077) and pseudo-destructor calls (an earlier oversight).

Replace construct/destruct interception by construct_at/destroy_at (and friends_.

Integrate San Diego CWG review notes.

Also rebase on N4778.

Introduction and motivation

Variable size container types, like `std::vector` or `std::unordered_map`, are generally useful for runtime programming, and therefore also potentially useful in constexpr computations. This has been made clear by some recent experiments such as the [Constexpr ALL the things!](#) presentation (and its companion paper [P0810R0](#) published in the pre-Albuquerque mailing) by Ben Deane and Jason Turner, in which they build a compile-time JSON parser and JSON value representation using constexpr. Amongst other things, the lack of variable size containers forces them to use primitive fixed-size data structures in the implementation, and to parse the input JSON string twice; once to determine the size of the data structures, and once to parse the JSON into those structures.

We also expect variable size containers to be a necessity in the reflection and metaprogramming APIs that will emerge from the work in SG-7, which decided that the preferred direction for a standard solution would involve constexpr-like computation. For example, querying the template arguments of a class type might look something like:

```
std::vector<std::metainfo> args = std::meta::get_template_args(reflexpr(T));
```

Non-transient allocation

Earlier versions of this paper ([P0784r1](#) and [P0784r2](#)) proposed the changes needed for constexpr destructors and for so-called "transient constexpr allocations". Transient constexpr allocations are dynamic memory allocations occurring during a constexpr evaluation that are deallocated before that evaluation completes.

What about dynamically allocated constexpr storage that hasn't been deallocated by the time evaluation completes? We could that, but there are really compelling use cases where this might be desirable. E.g., this could be the basis for a more flexible kind of "string literal" class. We therefore propose that a non-transient constexpr allocation be a valid result for a constexpr variable initializer if:

- the result of evaluating the initializer is an object with a nontrivial constexpr destructor, and
- evaluating that destructor would be a valid core constant expression and would deallocate all the *non-transient* allocations produced by the evaluation of *expr*.

Furthermore, we specify that an attempt to deallocate a non-transiently allocated object by any other means results in undefined behavior. (Note that this is unlikely because the object pointing to the allocated storage is immutable.)

A question that arises in this context is whether the non-transient allocation is mutable between the completion of the initialization and the evaluation of the destructor. If the allocation is mutable, reading from that allocation during the destructor evaluation is meaningless and should thus not be accepted as part of a core constant expression evaluation. However, there are cases where having a mutable allocation is desirable. So permit both cases, we introduce a library function "std::mark_immutable_if_constexpr" to designate that a constexpr allocation is immutable in the context.

Furthermore, we specify that an attempt to deallocate a non-transiently allocated object by any other means results in undefined behavior. (Note that this is unlikely because the object pointing to the allocated storage is immutable.)

For example:

```
#include <memory>
#include <new>
using namespace std;
template<typename T> struct S: allocator<T> {
    T *ps;
    int sz;
    template<int N> constexpr S(T (&p)[N])
        : sz{N}
        , ps{this->allocate(N)} {
        for (int k = 0; k<N; ++k) {
            new(this->ps+k) T{p[k]};
        }
        std::mark_immutable_if_constexpr(this->ps);
    }
    constexpr ~S() {
        for (int k = 0; k<this->sz; ++k) {
            (this->ps+k)->T::~~T();
        }
        this->deallocate(this->ps, this->sz);
    }
};

constexpr S<char> str("Hello!");
// str ends up pointing to a static array
// containing the string "Hello!".
```

The constructor constexpr evaluation in this example is successful, producing an S object that points to a non-transient constexpr allocation. The constexpr evaluation of the destructor would also be successful and would deallocate the non-transient allocation. The non-transient allocation is therefore promoted to static storage.

Virtual constexpr destructors

In Rapperswil, Peter Dimov proposed (P1077) to let literal types have virtual destructors. That proposal passed easily:

Allowing literal types to have virtual destructors:

SF: 11 | F: 12 | N: 2 | A: 0 | SA: 0

The notion of *quasi-trivial destructor* in P1077, however, is subsumed by this paper's notion of *constexpr destructor*. It was therefore decided to merge P1077 into this paper (with P1064, which permits virtual constexpr functions already approved by WG21 and edited into the current draft working paper).

Implementation experience

Constexpr destructors and support for transient dynamic allocation through the standard allocator has been implemented in the EDG compiler. Based on preliminary discussion with implementers working on Clang, MSVC and EDG, no blockers that would make this feature unimplementable or prohibitively expensive to implement have been identified.

Acknowledgments

Special thanks to Billy O'Neal for helping formulate the library wording changes.

Wording changes

Note 1: These are cumulative changes: They include the changes EWG approved for P0784r1, and build on top of that.

Note 2: The following changes enable "constexpr destructors". See further down for allocation-related changes.

Change in [basic.types] bullet 10.5.1:

- it has a ~~trivial~~constexpr destructor ([dcl.constexpr]),

Change in [expr.const] paragraph 2:

An expression *e* is a *core constant expression* unless the evaluation of *e*, following the rules of the abstract machine (6.8.1), would evaluate one of the following expressions:

- this (7.5.2), except in a constexpr function ~~or a constexpr constructor~~ ([dcl.constexpr]) that is being evaluated as part of *e*;
- an invocation of a ~~non-constexpr~~ function ~~other than a constexpr constructor for a literal class, a constexpr function, or an implicit invocation of a trivial destructor (10.3.7)~~ [*Note*: Overload resolution (11.3) is applied as usual — *end note*];
- an invocation of an undefined constexpr function ~~or an undefined constexpr constructor~~;
- an invocation of an instantiated constexpr function ~~or a constexpr constructor~~ that fails to satisfy the requirements for a constexpr function ~~or a constexpr constructor (10.1.5)~~;
- an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either
 - it is initialized with a constant expression or
 - its lifetime began within the evaluation of *e*;

— modification of an object (7.6.18, 7.6.1.6, 7.6.2.2) unless it is applied to a non-volatile lvalue of literal type that refers to non-volatile object whose lifetime began within the evaluation of *e*

Add new paragraph after [expr.const] paragraph 2:

An object *a* is said to have *constant destruction* if:

— it is not of class type nor (possibly multi-dimensional) array thereof, or

— it is of class type or (possibly multi-dimensional) array thereof, that class type has a `constexpr` destructor, and for a hypothetical expression *e* whose only effect is to destroy *a*, *e* would be a core constant expression if the lifetime of *a* and its non-mutable subobjects were considered to start within *e*.

Change in [dcl.constexpr] paragraph 2:

A `constexpr` specifier used in the declaration of a function ~~that is not a constructor~~ declares that function to be a *constexpr function*. ~~Similarly, a `constexpr` specifier used in a constructor declaration declares that constructor to be a *constexpr constructor*.~~

Change in [dcl.constexpr] paragraph 3 bullet 1:

its return type (if any) shall be a literal type;

Add a bullet to [dcl.constexpr] paragraph 3:

if the function is a constructor or destructor, its class shall not have any virtual base classes;

Change in [dcl.constexpr] paragraph 4:

The definition of a `constexpr` constructor whose *function-body* is not `= delete` shall additionally satisfy the following requirements:

- ~~the class shall not have any virtual base classes;~~
- ~~each of the parameter types shall be a literal type;~~
- ~~its function-body shall not be a function-try-block.~~

~~In addition, either its function-body shall be `= delete`, or it shall satisfy the following requirements:~~

- ~~either its function-body shall be `= default`, or the compound-statement of its function-body shall satisfy the requirements for a function-body of a `constexpr` function;~~
- every non-variant non-static data member and base class subobject shall be initialized (10.9.2);
- if the class is a union having variant members (10.4), exactly one of them shall be initialized;
- if the class is a union-like class, but is not a union, for each of its anonymous union members having variant members, exactly one of them shall be initialized;
- for a non-delegating constructor, every constructor selected to initialize non-static data members and base class subobjects shall be a `constexpr` constructor;
- for a delegating constructor, the target constructor shall be a `constexpr` constructor.

[Example: ...]

Insert new paragraph after [dcl.constexpr] paragraph 4:

The definition of a `constexpr` destructor whose *function-body* is not `= delete` shall additionally satisfy the following requirement:

— for every subobject of class type or (possibly multi-dimensional) array thereof, that class type shall have a constexpr destructor.

Change in [dcl.constexpr] paragraph 6:

If the instantiated template specialization of a constexpr function template or member function of a class template would fail to satisfy the requirements for a constexpr function ~~or a constexpr constructor~~, that specialization is still a constexpr function ~~or a constexpr constructor~~, even though a call to such a function cannot appear in a constant expression. If no specialization of the template would satisfy the requirements for a constexpr function ~~or a constexpr constructor~~, when considered as a non-template function ~~or constructor~~, the template is ill-formed, no diagnostic required.

Change in [dcl.constexpr] paragraph 8:

The constexpr specifier has no effect on the type of a constexpr function ~~or a constexpr constructor~~.

Change in [dcl.constexpr] paragraph 9:

In any constexpr variable declaration, the full-expression of the initialization shall be a constant expression (7.7). A constexpr variable shall have constant destruction.

Change in [class.dtor] paragraph 1:

Each decl-specifier of the decl-specifier-seq of a destructor declaration (if any) shall be friend, inline, ~~or virtual~~, or constexpr.

Add after [class.dtor] paragraph 9:

The defaulted destructor is a constexpr destructor if it satisfies the requirements for a constexpr destructor ([dcl.constexpr]).

Note 3: The following changes enable some "constexpr new-expressions".

Modify [expr.new] paragraph 10

An implementation is allowed to omit a call to a replaceable global allocation function (16.6.2.1, 16.6.2.2). When it does so, the storage is instead provided by the implementation or provided by extending the allocation of another *new-expression*.

~~The implementation may extend the allocation of a new-expression e1 to provide storage for a new-expression e2 if the following would be true were the allocation not extended:~~

- ~~— the evaluation of e1 is sequenced before the evaluation of e2, and~~
- ~~— e2 is evaluated whenever e1 obtains storage, and~~
- ~~— both e1 and e2 invoke the same replaceable global allocation function, and~~
- ~~— if the allocation function invoked by e1 and e2 is throwing, any exceptions thrown in the evaluation of either e1 or e2 would be first caught in the same handler, and~~
- ~~— the pointer values produced by e1 and e2 are operands to evaluated delete-expressions, and~~
- ~~— the evaluation of e2 is sequenced before the evaluation of the delete-expression whose operand is the pointer value produced by e1.~~

[Example:

...

— end example]

Add new paragraph after [expr.new] paragraph 10

During an evaluation of a constant expression, a call to an allocation function is always omitted. [Note: Only *new-expressions* that would otherwise result in a call to a replaceable global allocation function can be evaluated in constant expressions (see [expr.const]). — end note]

Add new paragraph after [expr.new] paragraph 10

The implementation may extend the allocation of a *new-expression* e1 to provide storage for a *new-expression*. e2 if the following would be true were the allocation not extended:

- the evaluation of e1 is sequenced before the evaluation of e2, and
- e2 is evaluated whenever e1 obtains storage, and
- both e1 and e2 invoke the same replaceable global allocation function, and
- if the allocation function invoked by e1 and e2 is throwing, any exceptions thrown in the evaluation of either e1 or e2 would be first caught in the same handler, and
- the pointer values produced by e1 and e2 are operands to evaluated *delete-expressions*, and
- the evaluation of e2 is sequenced before the evaluation of the *delete-expression* whose operand is the pointer value produced by e1.

[Example:

...

— end example]

Change in [expr.const] paragraph 2:

— a pseudo-destructor call (7.6.1.10);

...

— a *new-expression* (7.6.2.4);

— a *new-expression* (7.6.2.4), unless the selected allocation function is a replaceable global allocation function (16.6.2.1, 16.6.2.2) and either the allocated storage is deallocated within the evaluation of e or it is a *non-transient constexpr allocation* (see below);

— a *delete-expression* (7.6.2.5) unless it deallocates a region of storage allocated within the evaluation of e or it deallocates a *non-transient constexpr allocation* (see below);

— a call to an instance of `std::allocator::allocate` (`_allocator.members_`), unless either the allocated storage is deallocated within the evaluation of e or it is a *non-transient constexpr allocation* (see below);

— a call to an instance of `std::allocator::deallocate(_allocator.members_)`, unless it deallocates a region of storage allocated within the evaluation of `e` or it deallocates a *non-transient constexpr allocation* (see below);

Note 4: The following changes enable the use of the default allocator in constant expressions.

Add a new paragraph after [expr.const] paragraph 2:

For the purposes of determining whether an expression is a core constant expression, the evaluation of a call to a member function of `std::allocator<T>` as defined in `_allocator.members_`, where `T` is a literal type, does not disqualify the expression from being a core constant expression, even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression. Similarly, the evaluation of a call to `std::destroy_at`, `std::ranges::destroy_at`, `std::construct_at`, or `std::ranges::construct_at` is a valid core constant expression unless

- for a call to `std::construct_at` or `std::ranges::construct_at`, the first argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` or the evaluation of the underlying constructor call is not a core constant expression, or
- for a call to `std::destroy_at` or `std::ranges::destroy_at`, the first argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` or the evaluation of the underlying destructor call is not a core constant expression.

In [memory.syn] paragraph 1 and [specialized.destroy] paragraph 1 add the `constexpr` specifier to the all the declarations of `destroy_at`, `destroy`, and `destroy_n` (both in namespace `std` and in namespace `std::ranges`).

In [memory.syn] paragraph 1, after the declarations of `destroy` add declarations for `construct_at` as follows

```
template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args);

namespace ranges {
    template<class T, class... Args>
        requires Constructible<T, Args...>
            constexpr T* construct_at(T*, Args&&...);
}
```

and following [specialized.destroy] add a new subsection [specialized.construct] as follows:

```
template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args);

namespace ranges {
    template<class T, class... Args>
        requires Constructible<T, Args...>
            constexpr T* construct_at(T*, Args&&...);
}
```

Effects: Equivalent to:

```
return ::new (voidify(location)) T(std::forward<Args>(args)...)
(std::construct_at(p, std::forward<Args>(args)...)).
```

Modify [allocator.traits.members] paragraph 5 (about the `construct` member) as follows:

Effects: Calls `a.construct(p, std::forward<Args>(args)...) if that call is well-formed; otherwise, invokes ::new (static_cast<void*>(p)) T(std::forward<Args>(args)...) std::construct_at(p, std::forward<Args>(args)...) .`

Modify [allocator.traits.members] paragraph 6 (about the `destroy` member) as follows:

Effects: Calls `a.destroy(p)` if that call is well-formed; otherwise, invokes `p->~T()`
`std::destroy_at(p).`

In `[default.allocator]` and `[allocator.member]`, declare every member to be `constexpr`. In particular, add `constexpr` to the destructor, the copy assignment operator, and the `allocate` and `deallocate` members.

In `[allocator.globals]` declare both comparison operators to be `constexpr`.

Note 5: *The following changes enable non-transient allocations and prevent `constexpr` allocation leaks.*

Add a bullet after bullet (2.8) in `[expr.const]` paragraph 2:

...

— an lvalue-to-rvalue conversion (7.3.1) that is applied to a glvalue designating an object in storage `S` obtained by an invocation of an allocation function during the initialization of a `constexpr` variable `v` if `S` has not been marked as immutable by a call to `std::mark_immutable_if_constexpr` and the lvalue-to-rvalue conversion is applied during the evaluation of the hypothetical expression considered when determining whether `v` has constant destruction (`[expr.const]`);

Change in bullet (6.2) of `[expr.const]` paragraph 6 and add sub-bullets as follows:

- if the value is of pointer type, it contains
 - the address of an object with static storage duration; or the address past the end such an object (7.6.6), or
 - the address of a function, or
 - a null pointer value, or,
 - an address within or past the end of a non-transient `constexpr` allocation (as defined below) and if, for each object stored in the non-transient allocation, a prvalue core constant expression with the value of the object would be a constant expression,
- and

Add to end of `[expr.const]` paragraph 6:

A non-transient `constexpr` allocation is a region of storage `A` obtained by an invocation of an allocation function during the initialization of a `constexpr` variable `v` of class type or (possibly multi-dimensional) array thereof, but not passed to a deallocation function during that initialization, such that the hypothetical expression considered when determining whether `v` has constant destruction (`[expr.const]`) would deallocate `A`. Such a non-transient `constexpr` allocation is *immutable* if, during the initialization of the `constexpr` variable, `std::mark_immutable_if_constexpr` is called with an argument that is the value returned by the invocation of the allocation function. If `A` is deallocated except by the implicit invocation of `v`'s destructor, the behavior is undefined.

Add a bullet to `[temp.arg.nontype]` paragraph 2:

- a subobject (`[intro.object]`),
- an object stored in a non-transient `constexpr` allocation (`_expr.const_`),

Preceding `[ptr.laundry]` add a new subsection `[alloc.immutable]` as follows:

```
template<class T>
constexpr void mark_immutable_if_constexpr(T* p);
```

Effects: If called during the initialization of a `constexpr` variable with an argument that is a pointer value obtained by an invocation of an allocation function during that same initializer and that

allocation is a non-transient constexpr allocation, then that allocation is an immutable transient constexpr allocation (`_expr.const_`). Otherwise, no effect.